

STATS 21: Getting Set Up

Fall 2026 | Git, GitHub, Python, and VS Code

Miles Chen

This guide prepares your computer for lectures and homework. You will install the tools, connect to GitHub, make your own copy of the class repository, and run a notebook in the class Python environment.

Before you start: Create a [GitHub account](#), and make sure you can sign in. Use **Terminal on macOS** or **PowerShell on Windows** for commands in this guide. In VS Code on Windows, choose PowerShell as the terminal profile. Enter commands one at a time; a long command may wrap visually onto a second line.

You can keep an existing Python or Anaconda installation. This guide uses a separate class environment managed with **uv**. Follow the commands as written; you do not need to activate Conda's **base** environment.

1. Install and configure Git

macOS: Open Terminal and run `git --version`. If prompted to install Command Line Tools, finish that installation. If Git is still unavailable, run `xcode-select --install`, finish the installation, and open a new Terminal.

Windows: Install Git from git-scm.com/install/windows. Use the defaults; when asked for Git's default editor, choose Notepad instead of Vim. Close and reopen PowerShell afterward.

Checkpoint: Run the following. Success is a version number, such as `git version 2.x.x`, with no installation prompt or error.

```
git --version
```

Set the name and email attached to your commits. Replace the example values with your own; use an email associated with your GitHub account (your GitHub-provided private email is also fine).

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Checkpoint: These commands should print your intended name and email:

```
git config --get user.name
git config --get user.email
```

2. Install VS Code and uv

2.1 Visual Studio Code

1. Download **Visual Studio Code** from code.visualstudio.com. This is a different application from Microsoft Visual Studio.
2. **macOS:** Open the downloaded archive and drag Visual Studio Code to Applications. **Windows:** Run the installer and follow the default instructions.
3. Open VS Code. Click **Extensions** in the left sidebar.
4. Install **Python** and **Jupyter**, both published by **Microsoft**. Allow their supporting extensions to install as well.

Checkpoint: In Extensions, confirm that Python and Jupyter are installed and enabled. You do not need an Anaconda installation to use them.

2.2 uv: the class Python manager

uv will obtain Python, create a separate environment for this class, and install the required packages. Install uv using **only the command for your operating system**.

macOS - run in Terminal:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows - run in PowerShell:

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Close all Terminal or PowerShell windows. If VS Code is open, fully quit it too. Then open a new Terminal or PowerShell window. Restarting lets these programs find the newly installed tools.

Checkpoint: Both commands below should print version numbers:

```
uv --version  
git --version
```

If a command is not recognized: Make sure the installation finished successfully, then restart the application in which you are typing the command. If it still fails, ask for help before proceeding. Do not install another copy of Python to fix an unrecognized uv command.

Official installation reference: [Installing uv](#).

3. Connect this computer to GitHub

SSH keys allow your computer to authenticate to GitHub when downloading or uploading work. Each computer can have its own key. The **public** key ends in `.pub`; the matching file without `.pub` is private and must not be shared.

3.1 Check for an existing key

Run the command for your operating system:

- **macOS:** `ls -al ~/.ssh`
- **Windows PowerShell:** `Get-ChildItem -Force "$HOME\.ssh"`

If the folder does not exist, that is normal for a new setup. If you already have `id_ed25519` and `id_ed25519.pub`, you can reuse that pair for your GitHub account and continue to Section 3.3. If you have other keys or use another GitHub account, ask for help choosing the right key.

Do not overwrite an existing key. If a key-generation command asks whether to overwrite a file, answer `n` and stop.

3.2 Generate a key if you need one

Replace the example email with your own:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Accept the default save location by pressing `Enter`, provided no key already exists there.

Security best practices recommend protecting your SSH key with a passphrase. For this course, I personally recommend **leaving it blank for convenience**. Keep your laptop itself secure: use a strong login password and lock it when you step away.

When asked for a passphrase, **press `Enter` without typing anything**, then **press `Enter` again** to confirm the blank passphrase.

3.3 Copy and add the public key

macOS:

```
pbcopy < ~/.ssh/id_ed25519.pub
```

Windows PowerShell:

```
Get-Content "$HOME\.ssh\id_ed25519.pub" | Set-Clipboard
```

In GitHub, open [Settings > SSH and GPG keys](#). Choose **New SSH key**, give it a computer-specific title, leave the key type as **Authentication Key**, paste the public key, and choose **Add SSH key**. If the key is already listed for this account, you do not need to add it again.

4. Test GitHub and copy the class repository

4.1 Test SSH before cloning

```
ssh -T git@github.com
```

The first connection may ask whether you trust GitHub's host key. Compare the displayed fingerprint with [GitHub's published fingerprints](#) before entering yes.

Checkpoint: The response should identify **your GitHub username**, say authentication succeeded, and explain that GitHub does not provide shell access. The lack of shell access is normal. This successful test can return exit status 1. If you see **Permission denied (publickey)** or another account's username, fix that before continuing.

4.2 Fork and clone

1. Open [the Fall 2026 class repository](#).
2. Click **Fork** to create a copy in your own GitHub account. Keep the repository name `26-fall-stats-21`.
3. In Terminal or PowerShell, navigate to the **parent folder** where you want to keep classwork. For example, `cd ~/Documents` works if that folder exists on your computer. Do not clone inside another copy of the class repository.
4. Replace `your_github_name` below with your GitHub username, then run:

```
git clone git@github.com:your_github_name/26-fall-stats-21.git
cd 26-fall-stats-21
git remote add upstream git@github.com:smileschen/26-fall-stats-21.git
git config --local branch.main.rebase false
git config --local pull.ff true
git config --local branch.main.mergeOptions "--no-edit"
```

origin is your fork, where you save your work. **upstream** is the professor's repository, where you receive new course materials.

The last three commands configure this clone's **main** branch once. Pulls will fast-forward when possible, otherwise merge, and use the default merge message without opening an editor. Repeat this setup if you clone the repository on another computer.

Checkpoint: Run:

```
git rev-parse --show-toplevel
git branch --show-current
git remote -v
```

The first result should end in `26-fall-stats-21`. The branch should be **main**. Both **origin** and **upstream** should appear, usually with a fetch line and a push line for each. Check the **account names**, not just the repository names.

From this point on, run terminal commands from the **root of this class folder**, unless a step explicitly says otherwise. Clicking folders in VS Code's file explorer does not change your terminal's location.

5. Create the class Python environment

5.1 Open the class folder

In VS Code, choose **File > Open Folder** and select your cloned `26-fall-stats-21` folder. Trust this class folder if prompted. Choose **Terminal > New Terminal** (PowerShell on Windows).

Run `pwd`: it should show the class folder. Both requirements files should be visible in VS Code's file explorer.

5.2 Create the environment - first time only

```
uv venv --managed-python --python 3.12
```

This creates `.venv` in the class folder using Python managed by uv. An initial Python download is normal. **Do not rerun this command as a daily setup or a check:** recreating an environment can replace its installed packages.

Checkpoint - Windows PowerShell:

```
.\.venv\Scripts\python.exe -c "import sys; print(sys.version); print(sys.executable)"
```

Checkpoint - macOS:

```
.venv/bin/python -c "import sys; print(sys.version); print(sys.executable)"
```

Expect Python **3.12.x** and an executable path inside this class folder's `.venv`.

5.3 Install the packages

Run **only the command for your operating system**.

Windows:

```
uv pip install --python .venv -r requirements-win.txt
```

macOS:

```
uv pip install --python .venv -r requirements-unix.txt
```

Checkpoint: Run the following and confirm that it reports no missing or incompatible dependencies:

```
uv pip check --python .venv
```

6. Select Python and test a notebook

6.1 Select the editor's interpreter

Open the Command Palette: **Ctrl+Shift+P** on Windows or **Cmd+Shift+P** on macOS. Run **Python: Select Interpreter** and select the class folder's `.venv`.

If it is missing, choose **Enter interpreter path** and browse to `.venv\Scripts\python.exe` on Windows or `.venv/bin/python` on macOS. A popup offering to select the environment is convenient, but is not required. Terminal activation is optional: the package commands explicitly target `.venv` even if another environment is active.

6.2 Select the notebook's kernel

Create a new notebook with **Create: New Jupyter Notebook** in the Command Palette. Save it as `setup_check_my_copy.ipynb` in the class folder.

In the notebook's upper-right corner, choose **Select Kernel** (or click the currently selected kernel). If needed, choose **Select Another Kernel... > Python Environments**, then choose the class `.venv`. Check the full path; several environments may have similar names.

The notebook kernel and the editor interpreter are separate selections. Check the notebook kernel whenever you open a different notebook. If VS Code is in Restricted Mode, use **Manage Workspace Trust** to trust the class folder first.

6.3 Run this in a notebook code cell

Paste the following into a **notebook cell**, not the terminal. Run it with the play button or Shift+Enter.

```
import sys
from pathlib import Path

print("Python:", sys.version)
print("Interpreter:", sys.executable)
assert sys.version_info[:2] == (3, 12)
assert sys.prefix != sys.base_prefix
assert Path(sys.prefix).name == ".venv"

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy, seaborn, sklearn, polars
print("Core package imports passed.")
plt.plot([1, 2, 3], [1, 4, 9])
plt.show()
```

Checkpoint: The printed path must belong to **this class folder's `.venv`**, the success message must appear, and a plot must be visible. No red error message should remain. Save the notebook with Ctrl+S or Cmd+S.

7. Test saving work to your GitHub fork

Before the first class, complete one full save, commit, and push. This verifies that your setup works from the notebook all the way to GitHub.

With the terminal at the class repository root, run:

```
git status
git add setup_check_my_copy.ipynb
git status
```

Checkpoint: Under **Changes to be committed**, you should see only `setup_check_my_copy.ipynb`. The `.venv` folder should never be staged; the class `.gitignore` excludes it. If you see unexpected files, stop and ask for help before committing.

```
git commit -m "Complete setup check"
git push origin main
git status
```

Checkpoint: The push should succeed, and the final status should say **nothing to commit, working tree clean**. Open **your fork** on GitHub, refresh the page, and confirm that your saved setup notebook is present. A successful SSH test alone does not prove that your work has been uploaded.

8. Know where to work

The class uses these folders:

Folder	How to use it
<code>live_demos/</code>	Copy the day's starter notebook, rename the copy, and work in your copy. Keep the original unchanged.
<code>completed_notes/</code>	Read the professor's completed notes. If you want to run or edit them, work in a renamed copy.
<code>homework/</code>	Follow the assignment instructions and make your own copy of a starter notebook before editing.
<code>data/</code>	Use the course datasets. Keep provided files unchanged unless an assignment tells you otherwise.

Folders may appear as materials are added; Git does not store empty folders. Make copies beside the originals so that relative paths to datasets continue to work.

Running a notebook can change its saved outputs and metadata even if you do not change the code. Working in distinctly named copies reduces merge conflicts when the professor updates the originals.

Homework submission: Follow the assignment's submission instructions. A push to your fork is not automatically a homework submission. Files pushed to a public fork are public; do not upload homework solutions there unless the assignment permits it.

9. Daily class workflow

9.1 At the start of class

Open the class folder in VS Code. Save any open notebooks, then check:

```
git status
```

Checkpoint: Look for **nothing to commit, working tree clean**. If your own copies have changes, save and commit them using Section 9.2 before pulling. If an original course file is listed, use the troubleshooting guidance before continuing.

Get the latest materials:

```
git pull upstream main
```

The one-time configuration in Section 4.2 lets this command merge the professor's updates with your committed work. A merge commit can be normal even when there are no conflicts. Stop if Git reports a conflict or refuses to update; do not keep running later steps.

In `live_demos/`, copy the day's notebook and give it a distinct name. For example, copy `lecture_2.ipynb` to `lecture_2_my_copy.ipynb`. Open your copy, select the class `.venv` kernel, and work there.

9.2 After class

Save the notebook first with **Ctrl+S** or **Cmd+S**. Then run the following from the repository root, replacing the filename and message as appropriate:

```
git status
git add live_demos/lecture_2_my_copy.ipynb
git status
```

Checkpoint: Only files you intend to save should appear under **Changes to be committed**. The example path includes `live_demos/` because your terminal is at the repository root.

```
git commit -m "Add my notes for lecture 2"
git push origin main
git status
```

Checkpoint: The push should succeed and the final status should say **nothing to commit, working tree clean**. Confirm important work on your GitHub fork. If Git says there is nothing to commit, check that you saved the notebook; if nothing changed, a new commit is unnecessary.

Saving a file updates your computer. Committing records a local snapshot. Pushing uploads the commits to your GitHub fork.

10. Update packages and resolve setup problems

10.1 When the professor announces new packages

Save and commit your work, then pull updates using Section 9.1. From the class repository root, rerun the appropriate package command.

Windows:

```
uv pip install --python .venv -r requirements-win.txt
```

macOS:

```
uv pip install --python .venv -r requirements-unix.txt
```

Then run:

```
uv pip check --python .venv
```

These commands install or adjust packages to satisfy the updated requirements. They do not remove every extra package. Restart the notebook kernel after an update, then rerun the setup-check cell in Section 6.3.

Keep the class requirements files unchanged. Do not run `uv pip freeze` into either of them. You do not need to recreate `.venv` for a routine package update.

10.2 Common setup problems

A requirements file cannot be found: Run `pwd` and check that you are in the class repository root. Verify the spelling and use the correct operating-system filename. There is no class file named simply `requirements.txt`.

An import fails, or the notebook uses the wrong Python: Check the notebook's kernel and print `sys.executable` in a cell. Select the class `.venv`, rerun the correct installation command above, and restart the kernel. Installing packages into another Python will not repair this notebook.

PowerShell says `Activate.ps1` cannot run: This is a terminal-activation problem, not proof that environment creation failed. The explicit `.venv` commands in this guide and the selected notebook kernel can work without activation. Run the Section 5.2 checkpoint. If it passes, continue with the explicit package commands and kernel selection. Ask for help if you need terminal activation; changing the execution policy is not required for the main setup path.

The terminal prompt shows `(base)`: Anaconda has activated an environment. That label does not tell you which Python a notebook uses. Keep `--python .venv` in package commands and check the notebook's interpreter path. If Conda activation causes errors or the wrong interpreter keeps returning, ask for help with your existing installation.

SSH works in one terminal but Git still cannot authenticate: On Windows, different SSH programs may be in use. Retry from the same PowerShell session used for the SSH test, and ask for help with the exact error instead of generating more keys.

11. Recover from accidental edits carefully

If a pull says local changes would be overwritten, first inspect:

```
git status
```

Save useful notebook content into a distinctly named copy before discarding anything. **Do not run `git restore` as a general fix:** it can discard your own uncommitted work in files you previously committed.

11.1 Restore one accidentally edited original

Use the procedure below **only** for an original file whose changes you do not need, when those changes have not been staged or committed and no merge is in progress. In `git status`, unstaged edits appear under **Changes not staged for commit**. If you are unsure, ask for help.

For example, if the only accidental change is to the original `live_demos/lecture_2.ipynb`, first preserve anything useful, then run:

```
git restore -- live_demos/lecture_2.ipynb
git status
```

Replace the example path with the **specific original file** you intend to restore. This discards that file's unstaged edits; it does not download the professor's latest version.

Once your own work is safely committed and the working tree is clean, retry:

```
git pull upstream main
```

11.2 Cases that need separate help

Stop and ask for help if Git reports a merge conflict, an original file was already staged or committed, a push is rejected, or the status output is unclear. Avoid force-pushing or deleting the repository to try to fix the problem.

When asking for help, provide:

- Your operating system and whether you are using VS Code's terminal.
- The command you ran and the complete error message.
- The output of `pwd` and `git status` for Git or file-location problems.
- The printed interpreter path from Section 6.3 for Python or notebook problems.

Never share your private SSH key.

Setup is complete when: Git and uv report versions; SSH recognizes your account; the repository remotes are correct; Python runs from the class `.venv`; package checks and the notebook test pass; and your personal setup notebook appears on your GitHub fork.

Optional appendix: your own future projects

This appendix is for projects **outside the STATS 21 repository**. You do not need these steps to complete the class setup.

A. Create a separate project

Create a new project folder outside the class folder, open it in VS Code, and open a terminal in that new folder. Then run:

```
uv venv --managed-python --python 3.12
uv pip install --python .venv ipykernel pandas numpy matplotlib
```

Select this new project's interpreter and notebook kernel. Each project should have its own environment.

B. Record installed package versions

From your own project folder, use the command for your operating system.

Windows PowerShell:

```
uv pip freeze --python .venv | Set-Content -Encoding utf8 requirements.txt
```

macOS:

```
uv pip freeze --python .venv > requirements.txt
```

This records the installed package versions. Commit the requirements file with your project code, and exclude `.venv/` using your project's `.gitignore`.

Do not use this command to replace the class requirements files. A frozen package list records package versions; it does not capture every detail of the operating system, hardware, or Python installation.

C. Recreate a compatible environment

After obtaining a copy of your own project on another computer, open its folder and run:

```
uv venv --managed-python --python 3.12
uv pip install --python .venv -r requirements.txt
uv pip check --python .venv
```

This attempts to install the recorded versions on a compatible system. Some dependencies are specific to an operating system or processor, so a snapshot from one machine may need adjustment on another. Run the project's code to verify that it works.